

Accelerating Ultrasound Beamforming on Great Lakes HPC Cluster

Programming Assignment 1

Due date: 9/24/2025 11:59 PM

3D ultrasound is a growing area in medical imaging, but due to the high computation requirements, high frame rates and large imaging apertures, it is difficult to achieve without large cluster systems or specialized architectures. The computation itself is inherently simple but requires massive parallelism and optimized memory accesses to complete efficiently. The goal of this project is to explore how to map ultrasound processing to the Great Lakes HPC cluster to drastically improve performance over a sequential baseline.

The main aim of the assignment is to ensure all students in EECS 570 have some familiarity with parallel programming using pthreads and optimizing for multi-core CPU systems.

Programming Task

We have supplied you a sample program that performs delay-and-sum beamforming given pre-processed receive channel data from an array of ultrasound transducers. The program we have provided is sequential. Your task is to accelerate this program by parallelizing the computation on the Great Lakes HPC cluster using pthreads.

Along with the sample program, we have supplied three input files, `beamforming_input_{16,32,64}.bin`. These input files are located in the shared data directory on Great Lakes. These files contain transducer geometry, ultrasound image geometry, and pre-processed receive channel data for three different image resolutions (number of scanlines in the lateral image dimensions). The total amount of computation scales approximately quadratically in the number of scanlines, so the three inputs allow you to scale the runtime of the program. You should use the smallest input (16) for development and testing and then measure the final speedup of your solution on the largest input (64). You will be graded only on your performance on the largest input.

The supplied example program initializes various data structures, allocates memory, and then loads the input data from a file. The paths to these files are configured through environment variables; you select among the three inputs by specifying 16, 32, or 64 as a parameter to the binary.

Once the geometry and data is loaded, the computation proceeds in two steps. The first loop nest computes the distance from each transmitting transducer to each focal point in the image geometry, using Euclidean distance. The second loop nest calculates the distance from the focal point to each receiving transducer, sums the two distances, and then determines the index within the receive data that is nearest to the corresponding round trip time. This receive data element is then read from the rx_data array and added to the appropriate focal point in the image array.

The final image is then written to beamforming_output.bin. The output file can be compared against reference outputs using the solution_check program supplied along with the assignment, which checks the output file against a reference solution.

For the 64-scanline input, the baseline runtime of the computation phase of the unmodified sequential code we provided on a Great Lakes standard node is about 105 seconds. We will use this time as a baseline against which we measure your speedup. We will run your final submission at least three times and base your speedup on the median runtime.

Infrastructure

The University of Michigan Great Lakes HPC cluster provides access to [standard compute nodes](#) for this course. Each standard node features 36 physical cores with 187 GB of RAM, providing excellent opportunities for parallel programming and performance optimization.

To facilitate shared access and ensure it is possible for students to measure performance accurately, we use the SLURM batch job scheduler which provides exclusive access to compute resources during job execution.

You have been granted access to the eeecs570f25s001_class account on Great Lakes. This account has access to the standard partition with compute hours sufficient for all course work ([roughly 4000 CPU hours](#)). You may submit batch jobs for development, testing, and performance measurement using SLURM.

To enable reliable performance measurements, we have set up SLURM batch submission that can submit jobs to be run on [standard compute nodes](#). These jobs are capped at 5 minutes of runtime; if your job does not finish within 5 minutes, it will be killed.

The baseline code and submission script can be downloaded from the shared data directory on Great Lakes. For convenience, all files have been placed at /scratch/eeecs570f25s001_class_root/eeecs570f25s001_class/shared_data/PA1.

Getting Started

Step 1: Access Great Lakes

To access Great Lakes, use SSH to connect to the login node:

```
ssh your_username@gl-login1.arc-ts.umich.edu
```

Step 2: Set Up Your Project Directory

Create your project directory and copy the starter code:

```
cd /scratch/eecs570f25s001_class_root/eecs570f25s001_class/YOUR_UNIQNAME
mkdir PA1
cd PA1
cp -r /scratch/eecs570f25s001_class_root/eecs570f25s001_class/shared_data/PA1/code/* .
```

You should now have:

- beamform.c - Baseline sequential implementation
- solution_check.c - Output validation program
- submit.sh - SLURM job submission script

Step 3: Test with Batch Jobs (Recommended)

The recommended approach is to test by submitting batch jobs, as this is the same environment where your final performance will be measured. To test the baseline code:

```
sbatch submit.sh
```

The baseline code should produce output similar to:

```
Beginning computation
@@@ Elapsed time (usec): [baseline_time]
Processing complete. Preparing output.
Output complete: [output_path]
=== EECS 570 PA1 Solution Validation ===
Input size: 16 scanlines
Starting validation...
Validation complete!
RMS Error: 0.000000e+00
EXCELLENT! Output is correct (RMS error < 1e-16)
```

To interpret results:

- The "@@@ Elapsed time (usec):" line shows the computation time in microseconds
- The RMS Error should be less than 1e-16 for correct output
- The 16-scanline and 32-scanline inputs are useful for quick development and testing
- Measure final performance on the 64-scanline input for grading

Batch Job Submission

We have included a submission script, submit.sh, for the SLURM batch system. The script can be used to submit jobs to the standard partition on Great Lakes.

IMPORTANT: Your final beamform.c file MUST compile and run successfully with the EXACT submit.sh file provided. We will collect all students' final beamform.c files and test them using this exact submit.sh script. If your code does not compile or run with this script, you will receive no credit for the assignment. Additionally, your solution must be written in standard C. Inline assembly (__asm__) is strictly prohibited and will result in a grade of zero.

The submit.sh script automatically:

- Compiles your beamform.c using: `gcc -mavx2 -pthread -o beamform beamform.c -lm`
- Compiles the solution checker using: `gcc -o solution_check solution_check.c -lm`
- Sets the necessary environment variables
- Runs your beamforming program
- Validates the output using the solution checker

You configure which input file to use by setting the INPUT_SIZE variable in the script. The script automatically sets the necessary environment variables and submits the job to SLURM.

To submit a job, issue the command:

```
sbatch submit.sh
```

On Great Lakes, a free compute node is selected automatically by SLURM. Standard output is redirected to a file in your submission directory based on the job id assigned by SLURM. This will submit a job to the standard queue. You can see all your queued and running jobs with:

```
squeue -u $USER
```

See SLURM documentation for more information on how to manage queued jobs.

Local Development Option

While batch jobs are the recommended approach for testing on Great Lakes, you may also download the project files to your local machine for initial development and debugging. This can be helpful for:

- Initial code development and syntax checking
- Basic debugging with familiar development tools
- Testing small code changes before submitting to Great Lakes

Important: Local development is for convenience only. Your final performance measurements and grading will be based on runs on Great Lakes using the exact submit.sh script provided. Performance results from local machines will not be accepted for grading purposes.

Setting Up Local Development

To set up local development:

1. Download and extract the project zip file to your local machine
2. **Modify the source code paths:** Edit `beamform.c` and `solution_check.c` to change the hardcoded paths from Great Lakes to local paths:
 - In `beamform.c`: Change `input_dir` from Great Lakes path to `"/"`
 - In `solution_check.c`: Change `solution_dir` from Great Lakes path to `"/"`
 - Output paths automatically go to local `./outputs/` directory
3. Compile and test locally

Important: When you submit to Great Lakes, you must use the original unmodified source code with the Great Lakes hardcoded paths. The `submit.sh` script expects the original paths.

PThreads

Pthreads POSIX threads or Pthreads is a library implementation of a standardized C threads API that enables shared memory programming. In particular, the API provides two major functionalities -

- Thread Management: Functions for creating, detaching, joining threads etc...
- Synchronization: Mutexes, Conditional variables etc.

For more information on the Pthreads, check this [tutorial](#) from Lawrence Livermore National Labs.

Submission and Report

Submit a zip file containing your modified source file `beamform.c`. Also, submit a brief report (text or pdf) that describes your parallelization strategy (1 paragraph) and reports the runtime you measured on the 64-scanline input. The zip file should be submitted via Canvas.

Please upload a single file with the name <username>.zip containing `beamform.c` and your report file.

Grading

Your submission will be graded primarily based on (1) producing correct output on the 64-scanline input and (2) achieving a minimum speedup of 8x over the baseline single-threaded performance on Great Lakes. However, to encourage innovative attempts to use the various forms of parallelism available on the multi-core system, a small portion of your grade will be based on

how much *better* than 8x speedup your submissions achieves, scaled relative to the *best* performance of any submission. The grade breakdown will be:

- 40% - Correct output (RMS error < 1e-16 per solution_check)
- 50% - Speedup > 8x relative to sequential on Great Lakes
- 10% - Relative performance compared to the best submission

Collaboration

All programming assignments are to be completed individually. You may not share code with other students. We expect adherence to the Engineering Honor Code.

However, we encourage discussion of infrastructure issues, questions about the compiler or SLURM batching system, and related issues on Piazza. It is also acceptable to discuss parallelization strategies and code provided by the instructors as long as you do not exchange your own code.

Late Policy

No late submissions.

Project Files

The project files are available in the shared data directory on Great Lakes at `/scratch/eecs570f25s001_class_root/eecs570f25s001_class/shared_data/PA1/`.

For local development convenience, you can also download a zipped copy of the project files from [here](#). This zip contains all necessary files: `beamform.c`, `solution_check.c`, `submit.sh`, and all input/solution .bin files. Download this file to your local machine, extract it, and use it for development and testing. Remember that final performance measurements must be done on Great Lakes using the provided `submit.sh` script.

Resources

A sample parallel program using pthread is posted [here](#). (use "save as.." to download it). You can go through it and use any construct that you like.

[Introduction to Parallel Computing](#)

[POSIX Thread Tutorial](#)

[Great Lakes User Guide](#)

[SLURM Documentation](#)

A Word of Warning

As you can tell from this document, the Great Lakes HPC infrastructure is different from traditional development environments. We anticipate there will be infrastructure questions that have not been addressed in this document, and updates to the assignment or sample programs may be necessary. Please follow discussions on Piazza and start early.

Acknowledgements

The Great Lakes HPC cluster is provided by the University of Michigan Advanced Research Computing - Technology Services (ARC-TS). We thank ARC-TS for their support of this course.